

Reasoning about co-Büchi Tree Automata

Salvatore La Torre¹ and Aniello Murano^{1,2}

¹ Università degli Studi di Salerno

² Hebrew University

{slatorre,murano}@unisa.it

Abstract. We consider co-Büchi tree automata along with both alternating and generalized paradigms, as a characterization of the class of languages whose complement is accepted by generalized Büchi tree automata. We prove that the simulation theorem does not hold for alternating generalized co-Büchi tree automata and the generalized framework does not give more expressive power to this paradigm. We study the emptiness problem for the classes of languages whose complement is accepted by deterministic and nondeterministic generalized Büchi tree automata. For the nondeterministic case, we observe that the emptiness problem for alternating generalized co-Büchi tree automata is EXPTIME-complete. For deterministic case, we give a characterization in terms of generalized co-Büchi tree automata that yields a quadratic algorithm to decide the emptiness problem. Finally, we compare the classes of languages whose complement is respectively accepted by deterministic and nondeterministic Büchi tree automata with the main classes studied in the literature.

1 Introduction

Since its early days the theory of automata had an astonishing impact in computer science. Several models of automata have been extensively studied and applied to many fields. In the sixties, with their pioneering work, Büchi [Büc62], McNaughton [McN66], and Rabin [Rab69,Rab70] enriched this theory by introducing finite automata on infinite objects. Such automata turn out to be very useful for those areas of computer science where nonterminating computations are studied. They give a unifying paradigm to specify, verify, and synthesize nonterminating systems [Kur94,VW86,VW94]. A system specification can be translated to an automaton, and thus, questions about systems and their specifications are reduced to decision problems in the automata theory. More specifically, given a system S and its specification φ , an automaton A_S representing the system and an automaton $A_{\neg\varphi}$ accepting all computations that violate the specification, we can check the correctness of S with respect to φ by checking the emptiness of $A_S \times A_{\neg\varphi}$.

In system modelling, trees are more suitable than words to model nondeterminism, which is also useful to model concurrent programs (nondeterministic interleaving of atomic processes). It is worth noticing that some concurrent

programs, such as operating systems, communication protocols, and air-traffic control systems, are intrinsically nondeterministic and nonterminating. In the literature, several acceptance conditions on infinite words have been fruitfully extended to infinite trees: Büchi, co-Büchi, Muller, Rabin, Streett, and parity acceptance conditions (for more on these models see for example [GTW02]). As in Büchi automata, the co-Büchi paradigm uses a single set of final states. Here, we also consider these paradigms with respect to sets of final states (*generalized automata*). These richer acceptance conditions lead to automata with fewer states and simpler underlying structure. For example, the traditional translation of an LTL formula φ to a Büchi word automaton results in an automaton with $2^{\mathcal{O}(|\varphi| \times |\varphi|)}$ states [VW94], while using generalized automata we need only $2^{\mathcal{O}(|\varphi|)}$ states [GPVW95]. Generalized automata have become popular in specification and verification and now fruitfully used in several applications [Kur94]. The generalized co-Büchi condition was first introduced and studied on infinite words in [Lan69]. Its extension to infinite trees has been investigated in [LMN02].

The kind of acceptance condition influences both the closure properties and the complexity of the decision algorithms. For generalized Büchi and generalized co-Büchi tree automata non-emptiness is decidable in polynomial time [VW86,LMN02], for Rabin tree automata it is known to be NP-complete [Rab69]; on the other hand, generalized Büchi and generalized co-Büchi tree automata are not closed under language complementation, while Rabin and Muller tree automata are [Tho90].

Alternating tree automata along with the co-Büchi paradigm characterize the complement of languages nondeterministically accepted by Büchi tree automata [MS87]. Here, we consider these automata along with the generalized paradigm, namely, we consider alternating generalized co-Büchi tree automata (AGCTA), as a direct characterization of the class of languages whose complement is nondeterministically accepted by generalized Büchi tree automata (co-GBTA). In [MS87], it is shown that while alternation does not give more expressive power to Muller, parity, Rabin, Streett and Büchi paradigms (*simulation theorem*), it allows us to get more succinct automata. For example, translating an alternating Büchi tree automaton to a Büchi tree automaton might involve an exponential blow-up [MS95]. Once the simulation theorem holds, emptiness for alternating automata can be checked in an easy, and often efficient, way via translation to the corresponding nondeterministic model. For example, for an alternating Büchi tree automaton, we can construct a language equivalent Büchi tree automaton (which involves an exponential blow-up) and thus we can check for emptiness the starting automaton in exponential time which matches the known lower bound for the computational complexity of this problem.

Here, we prove that unfortunately the simulation theorem does not hold for AGCTA. In fact, we observe that generalized co-Büchi tree automata are not sufficiently powerful to characterize co-GBTA. We also prove that AGCTA and alternating co-Büchi tree automata (ACTA) are polynomially equivalent, that is, there exists a polynomial translation from an AGCTA to a language equivalent ACTA and viceversa. We observe that, when the generalized and the

corresponding non-generalized paradigms are language equivalent, the generalized one is still of interest since it can lead to more succinct automata with evident benefits in the design of efficient algorithms. As an example, we recall that nondeterministic generalized Büchi word automata and nondeterministic Büchi word automata are polynomially equivalent [Cho74], but computing the complement of a nondeterministic generalized Büchi automaton without constructing first the language equivalent nondeterministic Büchi automaton, may result in an automaton that is more succinct by an exponential factor [KV04].

Using the equivalence between AGCTA and ACTA, it follows that an AGCTA A can be translated to a parity tree automaton with two parity sets whose size is polynomial in the size of A . Thus, the emptiness problem for alternating generalized co-Büchi tree automata can be decided in exponential time. This result is also complete, since we can reduce the emptiness problem for weak alternating Büchi tree automata that is known to be EXPTIME-hard [KVV00].

To characterize the class of languages whose complement is accepted by generalized deterministic Büchi tree automata (co-DGBTA) we use the generalized co-Büchi paradigm along with the request that at least one path of an accepting run must be successful ($\exists$ -acceptance). This kind of “existential” acceptance differs from the usual request for tree automata that all paths need to be successful in order to accept. We recall that the $\exists$ -acceptance can be also seen as a restricted form of the Γ -acceptance in the sense of [BN95], where a run is successful if the accepting conditions are satisfied on a number of paths specified by the set Γ (*set of cardinality*). With respect to the emptiness problem, this “existential” acceptance condition is equivalent to consider the tree automaton as a word automaton: each transition is split into several transitions (one for each state successor). Thus, given a DGBTA A with n states and k accepting sets, we can construct a Büchi word automaton B with $\mathcal{O}(nk)$ states such that the language accepted by B is empty if and only if the complement of the language accepted by A is empty. Using the fact that for Büchi word automata the emptiness problem is decidable in linear time [EL85], checking the emptiness for co-DGBTA can be decided in quadratic time. We recall that an elegant characterization of co-DGBTA can be obtained via weak alternating Büchi tree automata. Unfortunately, this characterization gives an exponential-time algorithm to solving the emptiness problem for co-DGBTA, since the emptiness problem for weak alternating Büchi tree automata is EXPTIME-complete [KVV00].

2 Preliminaries

Let Σ be an alphabet, k be a positive integer, and $\text{DOM} = \{0, 1, \dots, k-1\}^*$, we define an infinite k -ary Σ -tree t as a map $t : \text{DOM} \rightarrow \Sigma$. In the following, unless differently stated, an infinite k -ary Σ -tree will be referred to simply as a tree. For each tree, the elements in DOM are the nodes of the tree and the empty word ε corresponds to the root. Let $u, v \in \text{DOM}$, we say that u *precedes* v , denoted as $u < v$, if there exists an x such that $v = ux$. Let $\pi \subseteq \text{DOM}$, π is a *path* of a tree t if it is a maximal subset of DOM linearly ordered by $<$. If π is a path of a tree

t , then t/π denotes the restriction of the function t to the set π . We say that a symbol $a \in \Sigma$ occurs infinitely often in t/π if there exists an infinite number of nodes $u \in \pi$ such that $t(u) = a$. The set of symbols that occur infinitely often in t/π is denoted $\text{Inf}(t/\pi)$.

Given a tree t and a node $u \in \text{DOM}$, we define the *subtree* of t rooted at u as the tree t_u such that $t_u(v) = t(uv)$ for $uv \in \text{DOM}$. Let Σ be a finite alphabet, we denote by T_Σ^ω the set of Σ -valued trees. A language is a subset of T_Σ^ω . Given a language $T \subseteq T_\Sigma^\omega$ we denote with $\overline{T}$ the complement of T , that is, $\overline{T} = T_\Sigma^\omega \setminus T$. In the following, we deal exclusively with binary trees ($\text{DOM} = \{0, 1\}^*$). All the results we obtain also hold for k -ary trees, where $k \geq 2$. According to the introduced notation, we use t_0 and t_1 to denote, respectively, subtrees of t rooted respectively at the left and at the right child of the root. Moreover, with π_0 we denote the leftmost path in a tree, that is $\pi_0 = \{0\}^*$.

A *finite automaton on infinite trees* (TA) is a tuple $A = \langle \Sigma, Q, Q_0, \delta, F \rangle$ where Σ is the input alphabet, $Q \neq \emptyset$ is a finite set of states, $Q_0 \subseteq Q$ is the set of initial states, $\delta \subseteq Q \times \Sigma \times Q \times Q$ is the transition relation, and F specifies the acceptance condition (a condition that defines a subset of Q^ω , we define several types of acceptance conditions below). Intuitively, each transition suggests a *nondeterministic* choice for the next configuration of the automaton. If $|Q_0| = 1$ and δ is a total function $\delta : Q \times \Sigma \rightarrow Q \times Q$, then A is a *deterministic* automaton (DTA). Given an input tree t , a *run* r of A on t is a Q -tree such that $r(\varepsilon) \in Q_0$, and $(r(u), t(u), r(u0), r(u1)) \in \delta$, for all $u \in \text{DOM}$. With $\text{Run}_A(t)$ we denote the set of runs of a TA A on a tree t . Clearly, if A is deterministic then $|\text{Run}_A(t)| = 1$. Different classes of languages are obtained defining different notions of *successful* run. A tree t is accepted by a TA A if there exists a successful run r of A on t , that is, r satisfies the acceptance condition on all the paths of t . We consider here the following conditions.

- A run r satisfies a *generalized Büchi* condition $F = \{F_1, \dots, F_k\} \subseteq 2^Q$ if for each path π of r , and for each set $F_i \in F$, $\text{Inf}(r/\pi) \cap F_i \neq \emptyset$.
- A run r satisfies a *generalized co-Büchi* condition $F = \{F_1, \dots, F_k\} \subseteq 2^Q$ if for each path π of r and for some $F_i \in F$, $\text{Inf}(r/\pi) \subseteq F_i$.
- A run r satisfies a *Muller* condition $F = \{F_1, \dots, F_k\} \subseteq 2^Q$ if for each path π of r , $\text{Inf}(r/\pi) \in F$.
- A run r satisfies a *Rabin* condition $F = \{(B_1, G_1), \dots, (B_k, G_k)\} \subseteq 2^{Q \times Q}$ if for each path π of r , there exists a pair $(B_i, G_i) \in F$ such that $\text{Inf}(r/\pi) \cap B_i = \emptyset$ and $\text{Inf}(r/\pi) \cap G_i \neq \emptyset$.
- A run r satisfies a *parity* condition $F = \{F_1, F_2, \dots, F_{2k}\}$, where the F_i 's form a partition of the set of states, if for each path π of r the minimal index i for which $\text{Inf}(r/\pi) \cap F_i \neq \emptyset$ is even.

In the following, we refer to the number k appearing in the acceptance condition as the *index* of the corresponding automaton. In particular, we refer to Büchi and co-Büchi conditions to denote the respective generalized paradigms with only one set of states forming the acceptance condition. With $T(A)$ we denote the language accepted by a TA A , that is, the set of accepted trees.

To denote the different types of tree automata, we will use acronyms composed picking at most a letter from each of the following sets $\{D\}$, $\{G\}$, $\{B, C, M, R, P\}$. Letters are placed in the order we have listed the sets and the letter from the first two sets can be omitted. The first letter stands for deterministic (D), and omitting it we mean nondeterministic. Analogously, the second letter stands for generalized (G) and to denote that only a set forms the acceptance condition (clearly, G is only used for Büchi and co-Büchi paradigms). The third letter stands for denoting the type of the acceptance condition: Büchi (B), co-Büchi (C), Muller (M), Rabin (R) and parity (P). For example, deterministic co-Büchi tree automata are denoted by DCTA. We also use the same acronyms to denote the corresponding class of accepted languages.

Figure 1 recalls the known relationships between all the considered classes of tree languages (automata) [Rab70,LMN02,Tho90,GTW02]. Since the class of languages accepted by Rabin and parity tree automata coincides with the class of languages accepted by Muller tree automata, we only use MTA for this class, as well as DMTA for the deterministic one. For the same reason, we only use (D)BTA to denote the class of languages accepted by (deterministic) generalized Büchi tree automata with index $k \geq 1$.

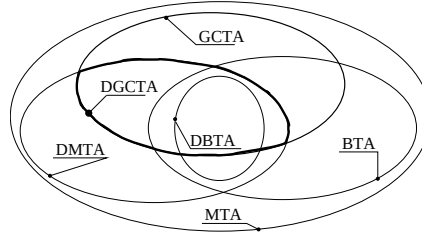


Fig. 1. Relationships between (D)BTA, (D)MTA and (D)GCTA.

The following theorem summarizes the closure properties of the above classes of automata and languages [Tho90,LMN02].

Theorem 1.

- *DMTA, DGBTA, and DGCTA are closed under intersection, but they are not closed under union and complementation.*
- *GBTA and GCTA are closed under intersection and union, but they are not closed under complementation.*
- *MTA is closed under intersection, union, and complementation.*

In the following theorem, we recall some known results on the decision problems for Büchi, generalized co-Büchi, Rabin, and parity tree automata.

Theorem 2.

- *The emptiness problem for BTA is decidable [Rab70], and is LOGSPACE-complete for PTIME[VW86].*

- The emptiness problem for GCTA is decidable and is in PTIME [LMN02].
- The emptiness problem for PTA is $UP \cap co-UP$ [Jur98]³.
- The non-emptiness problem for RTA is NP-complete [Rab69,EJ88].

3 Alternating tree automata

Alternating automata generalize the notion of nondeterminism by allowing several successor states along the same branch of the tree [MS87]. Muller and Schupp were the first to apply to tree automata the concept of alternation, introduced by Chandra, Kozen, and Stockmeyer [CKS81]. Here we briefly recall the basic definitions and refer to [MS95] for more details.

An *alternating tree automaton* is a TA with the transition relation defined as a function $\delta : Q \times \Sigma \rightarrow \mathcal{B}^+(K \times Q)$, where K is the set of *direction* in the tree ($K = \{0, 1\}$, for binary trees) and $\mathcal{B}^+(K \times Q)$ is the set of all positive boolean combinations of pairs (d, q) , where d is a direction and q is a state.

A run for an alternating automaton on a binary tree t is a $(\{0, 1\}^* \times Q)$ -labeled (possibly non binary) tree such that the root is labelled (ε, q_0) and labels of each node and its successors must satisfy the transition relation δ . For example, if $t(\varepsilon) = a$ and $\delta(q_0, a) = ((0, q_1) \vee (0, q_2)) \wedge ((0, q_3) \vee (1, q_2))$, then, a run r on t at level 1 must include a node labeled $(0, q_1)$ or a node labeled $(0, q_2)$, and must include a node labeled $(0, q_3)$ or a node labelled $(1, q_2)$.

As for standard tree automata, we can couple different acceptance conditions to an alternating tree automaton, defining different classes of languages and automata. To denote alternating automata, we use a prefix “A” to the acronyms used so far. For example, we use ABTA to denote alternating Büchi tree automata, as well as the class of languages accepted by these automata.

In [Cho74], it has been proved that GBTA and BTA are polynomially equivalent. In the next lemma, we extend this result to the alternating paradigm. That is, given an AGBTA with m states and index k , we can build a language equivalent ABTA with $\mathcal{O}(m(k + 1))$ states.

Lemma 1. *Given a AGBTA A , there exists an ABTA A' accepting $T(A)$ and whose size is polynomial in the size of A .*

Proof. Let $A = \langle Q, \Sigma, \delta, Q_0, \{F_1, \dots, F_k\} \rangle$ be an AGBTA. Consider $A' = \langle Q \times \{0, \dots, k\}, \Sigma, \delta', Q_0 \times \{0\}, Q \times \{k\} \rangle$ as an ABTA, such that, for each formula $\delta(q, \sigma)$ in A , the automaton A' contains a formula $\delta'(q, i, \sigma)$ obtained from $\delta(q, \sigma)$ by coupling each pair (q', d) in $\delta(q, \sigma)$ with a value j as follows: (i) $j=0$ if $i = k$, (ii) $j = i + 1$ if $q \in F_j$, or (iii) $j = i$ otherwise. Thus, A' enters an accepting state if at least one state for all accepting sets from A has been visited infinitely often. Thus, $T(A') = T(A)$ and the size of A' is polynomial in the size of A . $\square$

In [MS87], Muller and Schupp introduced *weak alternating Büchi tree automata* (WABTA) as a special case for ABTA. In a WABTA, we have a Büchi

³ The class UP is a subset of NP, where each word accepted by the Turing machine has a unique accepting run

acceptance condition $F \subseteq Q$ and there exists a partition of Q into disjoint sets, $Q_1, \dots, Q_m$, such that for each set Q_i , either $Q_i \subseteq F$, in which case Q_i is an *accepting* set, or $Q_i \cap F = \emptyset$, in which case Q_i is a *rejecting* set. In addition, there exists a partial order $\leq$ on the collection of the Q_i 's such that for every $q \in Q_i$ and $q' \in Q_j$ for which $\delta(q, \sigma, q', q'')$ or $\delta(q, \sigma, q'', q')$ occur, we have $Q_j \leq Q_i$. Thus, transitions from a state in Q_i lead to states in either the same Q_i or a lower one. It follows that every infinite path of a run of a WABTA ultimately gets “trapped” within some Q_i . The path then satisfies the acceptance condition if and only if Q_i is an accepting set.

The main properties about weak alternating Büchi, alternating Büchi, and alternating parity tree automata are summarized in the following theorem. We recall that an alternating automaton is deterministic if and only if the transition relation δ does not use $\vee$ [MS95].

Theorem 3.

- Given an $A(D)BTA$ (resp., an $A(D)PTA$) A , there exists a $(D)BTA$ (resp., a $(D)PTA$) accepting $T(A)$, whose size is exponential in the size of A [MS87].
- The emptiness problem for $(W)ABTA$ is EXPTIME-complete [KVV00, MS87].
- The emptiness problem for $APTA$ is in EXPTIME [EJ91, Wil01].

Directly from Lemma 1 and Theorem 3 we also get the following result.

Corollary 1. *The emptiness problem for AGBTA is decidable in exponential time.*

As discussed in [MS87], an advantage of using alternation is that one can complement an alternating automaton by dualizing its transition function and acceptance condition. Formally, given a transition function δ , let $\tilde{\delta}$ denote the dual function of δ . That is, for every $\varphi \in \delta$, we have $\tilde{\varphi} \in \tilde{\delta}$, where $\tilde{\varphi}$ is obtained by φ switching $\vee$ and $\wedge$ and by switching *true* and *false*. The dual of an acceptance condition F , denoted as $\tilde{F}$, is a condition that holds exactly on all the runs on which F does not hold. In particular, by denoting with $\tilde{A} = \langle Q, \Sigma, \tilde{\delta}, Q_0, \tilde{F} \rangle$ the dual automaton of an automaton $A = \langle Q, \Sigma, \delta, Q_0, F \rangle$, the following holds.

Theorem 4. [MS87] For an ABTA A , the ACTA $\tilde{A}$ accepts $\overline{T(A)}$, and viceversa.

4 Alternating generalized co-Büchi tree automata

In this section, we deal with alternating tree automata along with the generalized co-Büchi paradigm (AGCTA, for short). The definition of duality given in the previous section, along with the result shown in Theorem 4 makes this class a suitable choice for a direct characterization of the class of languages whose complement is in AGBTA, as pointed out in the following.

Corollary 2. *Given an AGBTA A , its dual $\tilde{A}$ is an AGCTA accepting $\overline{T(A)}$, and viceversa, given an AGCTA C , its dual $\tilde{C}$ is an AGBTA accepting $\overline{T(C)}$.*

Since A(G)CTA accepts the class of languages whose complement is accepted by (G)BTA, in the following, we also denote the class of languages accepted by A(G)CTA as co-(G)BTA. Lemma 2 shows that the class of languages accepted by AGCTA is polynomially equivalent to that one accepted by ACTA.

Lemma 2. *Given an AGCTA A , there exists an ACTA A' accepting $T(A)$ and whose size is polynomial in the size of A .*

Proof. Let A be an AGCTA, from Corollary 2, it follows that there exists an AGBTA B such that $\overline{T(B)} = T(A)$. From Lemma 1, it is possible to build an ABTA B' whose size is polynomial in the size of B such that $\overline{T(B)} = T(B')$. From Theorem 4, there exists an ACTA A' dual to B such that $\overline{T(B')} = T(A')$. Thus, $T(A') = \overline{T(B')} = \overline{T(B)} = T(A)$ and the size of A' is polynomial in the size of A , from the definition of duality. $\square$

The above lemma is useful to show the following result.

Theorem 5. *The emptiness problem for AGCTA is EXPTIME-complete.*

Proof. We first show that given an AGCTA A there exists an APTA B accepting $T(A)$ and whose size is polynomial in the size of A . From Lemma 2, we first translate A in an ACTA A' , whose size is polynomial in the size of A . Let $A' = \langle Q, \Sigma, \delta, Q_0, \{F\} \rangle$ be the obtained ACTA. An APTA accepting $T(A')$ is the automaton $B = \langle Q, \Sigma, \delta, Q_0, \{Q \setminus F, F\} \rangle$. Thus, membership to EXPTIME of the emptiness problem for AGCTA follows from the fact that the size of B is linear in the size of A' , the size of A' is polynomial in the size of A , and the emptiness problem for APTA is in EXPTIME (see Theorem 3).

It remains to prove the lower bound. For this purpose, we use a reduction from the emptiness problem for weak alternating Büchi tree automata, known to be EXPTIME-hard [KVV00]. Indeed, it is easy to check that a WABTA can be seen both as an alternating Büchi and an alternating co-Büchi tree automaton. In more details, given a WABTA A , it turns out that A accepts a language L if and only if the automaton A used as an alternating co-Büchi tree automaton B accepts the same language. Therefore, $L(A)$ is empty if and only if $L(B)$ is empty. Thus, the emptiness problem for AGCTA is EXPTIME-hard. $\square$

Directly from the above result, we also obtain the following.

Corollary 3. *The universality problem for GBTA is EXPTIME-complete.*

Proof. The upper bound follows from Corollary 2 and from the fact that the emptiness problem for AGCTA is decidable in exponential time (Theorem 5). For the lower bound, we notice that the universality problem for automata on finite trees is EXPTIME-hard [Sei90]. $\square$

We now study the relationships between co-BTA and the classes of languages we have introduced in the previous sections. For this purpose, in Figure 2 we list some languages along with their ranking relatively to the classification illustrated in Figure 1. For more details see [LMN02]. For all these languages we assume that $\Sigma = \{a, b\}$. Since the classes of languages co-GBTA, co-BTA, AGCTA, and ACTA coincide, in the following we only use co-BTA to refer to this class.

Languages	Ranking
$T_1 = \{t \in T_\Sigma^\omega \mid \exists \pi, \text{ either } a \notin \text{Inf}(t/\pi) \text{ or } b \notin \text{Inf}(t/\pi)\}$	$(\text{GCTA} \cap \text{BTA}) \setminus \text{DMTA}$
$\overline{T}_1 = \{t \in T_\Sigma^\omega \mid \forall \pi, a \in \text{Inf}(t/\pi) \text{ and } b \in \text{Inf}(t/\pi)\}$	$\text{DBTA} \setminus \text{GCTA}$
$T_2 = \{t \in T_\Sigma^\omega \mid \forall \pi, \text{ either } a \notin \text{Inf}(t/\pi) \text{ or } b \notin \text{Inf}(t/\pi)\}$	$\text{DGCTA} \setminus \text{BTA}$
$\overline{T}_2 = \{t \in T_\Sigma^\omega \mid \exists \pi, a \in \text{Inf}(t/\pi) \text{ and } b \in \text{Inf}(t/\pi)\}$	$\text{BTA} \setminus \text{GCTA}$
$T_3 = \{t \in T_\Sigma^\omega \mid a \notin \text{Inf}(t/\pi_0)\}$	$(\text{BTA} \cap \text{GCTA} \cap \text{DMTA}) \setminus \text{DBTA}$
$\overline{T}_3 = \{t \in T_\Sigma^\omega \mid a \in \text{Inf}(t/\pi_0)\}$	DBTA

Fig. 2. Some tree languages and their classification [LMN02].

Lemma 3. $\text{co-BTA} \not\subseteq \text{GCTA}$.

Proof. This result can be shown using the languages T_1 and $\overline{T}_1$. From the table in Figure 2, we know that $T_1 \in \text{BTA}$, thus $\overline{T}_1 \in \text{co-BTA}$, while $\overline{T}_1 \notin \text{GCTA}$. $\square$

Directly from the non-equivalence between generalized co-Büchi and alternating generalized co-Büchi paradigms showed in Lemma 3, we get the following important result for the latter.

Theorem 6. *The simulation theorem does not hold for alternating generalized co-Büchi tree automata.*

The following lemma states the results of all the remaining comparisons involving the class co-BTA. The complete picture of the relationships among all discussed classes is given in Figure 3.

Lemma 4.

1. $\text{GCTA} \subset \text{co-BTA}$.
2. $\text{DMTA} \subset \text{co-BTA}$.
3. BTA and co-BTA are not comparable.
4. $\text{co-BTA} \not\subseteq (\text{GCTA} \cup \text{BTA} \cup \text{DMTA})$.
5. $(\text{BTA} \setminus (\text{GCTA} \cup \text{DMTA})) \cap \text{co-BTA} \neq \emptyset$.
6. $\text{BTA} \cup \text{co-BTA} \subset \text{MTA}$.

Proof. To prove that $\text{GCTA} \subseteq \text{co-BTA}$, we recall that any GCTA A is also an AGCTA, and thus, from Theorem 4, $T(A)$ is the complement of a language accepted by the AGBTA B dual of A . Thus, the result follows from the fact that the generalized Büchi paradigm is equivalent to Büchi. Strict containment is a consequence of Lemma 3. Thus part 1 holds.

To prove that $\text{DMTA} \subseteq \text{co-BTA}$, we first notice that given a DMTA M and a tree t , M can only check that the acceptance condition holds on a fixed path π of t , or on all paths of t . Thus, $\overline{T}(M)$ consists of all trees such that the acceptance condition of M does not hold on π , in the first case, or on a nondeterministically selected path of t , in the second case. Since a nondeterministic selection can be easily done in BTA, and since on a single path the deterministic Muller paradigm is equivalent to the nondeterministic Büchi one (see [Tho90]), we conclude that $\overline{T}(M)$ is in BTA, thus, $T(M)$ is in co-BTA. Moreover, for $\text{GCTA} \subseteq \text{co-BTA}$,

$\text{DMTA} \subseteq \text{co-BTA}$, and the fact that GCTA and MTA are not comparable (see Figure 1), we get that part 2 holds.

Part 3 follows directly from the non-closure under complementation of BTA (Theorem 1). Finally, to prove parts 4, 5, and 6 we can respectively use the languages $\{t \in T_\Sigma^\omega \mid t_0 \in T_1 \text{ and } t_{10} \in T_2 \text{ and } t_{11} \in \overline{T_1}\}$, $\{t \in T_\Sigma^\omega \mid t_0 \in T_1 \text{ and } t_1 \in \overline{T_1}\}$ and $\{t \in T_\Sigma^\omega \mid t_0 \in \overline{T_2} \text{ and } t_1 \in T_2\}$, where T_1, T_2, T_3 , and their complements are given in Figure 2. $\square$

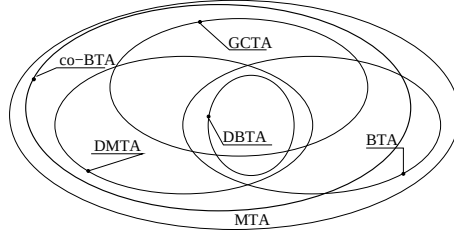


Fig. 3. Summary of the comparisons involving co-BTA .

5 co-DGBTA

In this section, we deal with the class of languages whose complement is deterministically accepted by generalized Büchi tree automata (co-DGBTA , for short). We study the relationships of co-DGBTA with the other classes introduced so far and the complexity of the emptiness problem. Clearly, from the results obtained in the previous sections, the fact that DGBTA is polynomially equivalent to DBTA , and the fact that DBTA is a special case of BTA , it follows that the emptiness problem for co-DGBTA can be solved in exponential time. Here, we prove that this problem is indeed decidable in polynomial time.

The first example of class within BTA closed under complementation has been the remarkable characterization by Rabin [Rab72] of languages defined by a formula of weak monadic logic (where only quantifiers over finite sets are allowed): A language T is *weakly definable* if and only if both T and its complement $\overline{T}$ are accepted by Büchi tree automata. In [MS87], it has been proved that a language is weakly definable if and only if it is accepted by a weak alternating Büchi tree automaton. The class of languages accepted by these automata includes co-DBTA . In general, we have the following strict containment.

Lemma 5. $\text{DBTA} \cup \text{co-DBTA} \subset \text{WABTA}$.

Proof. Let $T = \{t \in T_\Sigma^\omega \mid t_0 \in T_3 \text{ and } t_1 \in \overline{T_3}\}$, where T_3 and $\overline{T_3}$ are given in Figure 2. Since T_3 is not in DBTA , it follows that T is not in $\text{DBTA} \cup \text{co-DBTA}$. On the other hand, since both T_3 and $\overline{T_3}$ are in BTA , it follows that T is in WABTA . Hence, $T \in \text{WABTA} \setminus (\text{DBTA} \cup \text{co-DBTA})$. $\square$

Recall that an alternating automata is deterministic if and only if the transition relation δ does not use $\vee$ [MS95]. Directly from this definition and from the fact that complementing a WABTA by dualization gives an automaton with the same paradigm [MS87], we get the following characterization for co-DBTA.

Corollary 4. *co-DBTA is represented by WABTA using only $\vee$ in the transition relation.*

From Theorem 3, it follows that the emptiness problem for co-DBTA with the characterization given by Corollary 4 can be solved in exponential time. As we show in the following, this complexity can be reduced to a polynomial if we use a direct approach. First observe that a tree is not accepted by a DGBTA A if and only if the unique run r of A on t contains a path π that does not satisfy the acceptance condition. Thus, it is possible to characterize the complement of a language accepted by a DGBTA A with an automaton that, for each tree, nondeterministically selects a path and then deterministically checks that π does not satisfy the acceptance condition of A . This last corresponds to check that π satisfies the generalized co-Büchi condition obtained dualizing the acceptance condition of A . Thus, we modify the definition of accepting run. Given a tree t and a GCTA B with an accepting condition $F = \{F_1, \dots, F_k\}$, we say that a run $r \in \text{Run}_B(t)$ is $\exists$ -successful if there exists a path π of r , such that $\text{Inf}(r/\pi) \subseteq F_i$ for some $F_i \in F$. A tree t is $\exists$ -accepted by B if there exists an $\exists$ -successful run of B on t . The language $\exists$ -accepted by B is denoted by $T_\exists(B)$. In the next lemma, we show that the $\exists$ -acceptance along with the generalized co-Büchi paradigm suffices to accept co-DGBTA.

Lemma 6. *Given a DGBTA B , there exists a DGCTA A such that $T_\exists(A) = \overline{T(B)}$. Moreover, if B is DBTA then A is DCTA.*

Proof. Let T be a language whose complement is accepted by a DGBTA $B = \langle Q, \Sigma, \delta, Q_0, \{F_1, \dots, F_k\} \rangle$. Let $A = \langle Q, \Sigma, \delta, Q_0, \{Q \setminus F_i \mid i = 1, \dots, k\} \rangle$ be a DGCTA. A tree $t \notin T(B)$ if and only if the only run r in $\text{Run}_B(t)$ (since B is deterministic) is not successful. That is, r contains at least a path π such that $\text{Inf}(r/\pi) \cap F_i = \emptyset$, for some i . Thus, $t \notin T(B)$ if and only if $t \in T_\exists(A)$. $\square$

With respect to the emptiness problem, notice that the characterization of co-DGBTA via “existential” tree automata is equivalent to consider tree automata as word automata⁴. In more details, given a DGBTA B , consider a generalized co-Büchi word automaton C that is obtained from B by dualizing the acceptance condition and splitting each transition into several transitions, one for each state successor. That is, for each transition (s, σ, s', s'') of B , we get two transitions (s, σ, s') and (s, σ, s'') of C . It is easy to verify that there is a tree that is not accepted by B if and only if there is a word accepted by C . Using this observation, we get an efficient algorithm for solving the emptiness problem for co-DGBTA. First, we notice that a generalized co-Büchi word automaton

⁴ Tree automata generalize word automata, in the sense that a word is a tree of arity 1. Thus, we omit a formal definition of word automata here.

can be easily translated into a language equivalent generalized Büchi word automaton whose size is linear in the size of the starting automaton. Thus, given a DGBTA B with n states and k accepting sets, we can construct a Büchi word automaton A with $\mathcal{O}(nk)$ states such that the language accepted by A is empty if and only if the complement of the language accepted by B is empty. Since the emptiness problem for Büchi word automata is decidable in linear time [EL85], we get that checking for the emptiness of $\overline{T(B)}$ can be done in $\mathcal{O}(nk)$ time. Thus, the following theorem holds.

Theorem 7. *Given a DGBTA B with n states and index k , checking if $\overline{T(B)}$ is empty can be done in $\mathcal{O}(nk)$ time.*

The rest of the section is devoted to compare co-DGBTA with the other classes considered in this paper.

Lemma 7. *Given a DGBTA B , there exists a GCTA A such that $T(A) = \overline{T(B)}$. Moreover, if B is DBTA then A is CTA.*

Proof. Let T be a language whose complement is accepted by a DGBTA $B = \langle Q, \Sigma, \delta, Q_0, \{F_1, \dots, F_k\} \rangle$. We build a GCTA A that nondeterministically selects a path and on this path checks that the acceptance condition of B does not hold. Formally, $A = \langle \{q\} \cup Q, \Sigma, \delta', Q_0, \{\{q\} \cup Q \setminus F_i \mid i = 1, \dots, k\} \rangle$ be a GCTA, where $q \notin Q$ and δ' is defined as follows. For each $(s, \sigma, s', s'') \in \delta$, the transition relation δ' contains (s, σ, q, s'') and (s, σ, s', q) ; moreover, δ' contains (q, σ, q, q) . A tree $t \notin T(B)$ if and only if the only run r in $Run_B(t)$ (we recall that B is deterministic) is not successful. That is, r contains at least a path π such that $Inf(r/\pi) \cap F_i = \emptyset$, for some i . Thus, there exists a run r in $Run_A(t)$ such that for each path π , either $Inf(r/\pi) = \{q\}$ or there is an i such that $Inf(r/\pi) \subseteq Q \setminus F_i$. Hence, $t \notin T(B)$ if and only if $t \in T(A)$. $\square$

From the above construction, notice that co-DGBTA can be linearly characterized by GCTA. In the next lemma, we show that co-DGBTA can be polynomially characterized by BTA (notice that it is linear starting from co-DBTA).

Lemma 8. *Given a DGBTA B , there exists a BTA A accepting $\overline{T(B)}$, whose size is polynomial in the size of B .*

Proof. By [Cho74], we can restrict to DBTA. Let T be a language whose complement is accepted by a DBTA $B = \langle Q, \Sigma, \delta, Q_0, F \rangle$. We build a BTA A that nondeterministically selects a path and on this path checks that the acceptance condition of B does not hold. Formally, $A = \langle Q', \Sigma, \delta', Q'_0, F' \rangle$ is such that (i) $Q' = Q \times \{0, 1, 2\}$; (ii) $Q'_0 = Q_0 \times \{0\}$; (iii) $F' = Q \times \{1, 2\}$; (iv) if $(s, \sigma, s', s'') \in \delta$, the transition relation δ' contains: $((s, 0), \sigma, (s', 0), (s'', 1))$, $((s, 0), \sigma, (s', 1), (s'', 0))$, $((s, 1), \sigma, (s', 1), (s'', 1))$, $((s, h), \sigma, (s', 2), (s'', 1))$ for $h \in \{0, 2\}$ and $s' \in Q \setminus F$, and $((s, h), \sigma, (s', 1), (s'', 2))$ for $h \in \{0, 2\}$ and $s'' \in Q \setminus F$. First, observe that size of A is linear in the size of B . Moreover, A accepts a tree t if and only if, for the only run r of B on t (since B is deterministic), there exists a path π of r on which final states of B occur only finitely often. This is done by nondeterministically selecting the path π and then checking that this property holds for this path. Thus, $\overline{T(B)} = T(A)$, and the lemma is shown. $\square$

Let us observe that the characterizations of co-DGBTA given in Lemmas 7 and 8 allow to decide the emptiness problem for co-DGBTA via a reduction to the same problem for GCTA and BTA. Let B be a DGBTA with n states and index k . Since the best known upper bounds on the time complexity of the emptiness problem for GBTA and GCTA are both quadratic in the number of states and linear in the index of the automaton [Rab70,LMN02], the time complexities of these algorithms are greater than that given in Theorem 7.

In the following lemmas, we complete the comparisons involving co-DBTA and the classes GCTA, (D)BTA and DMTA. The complete picture of the comparisons is given in Figure 4.

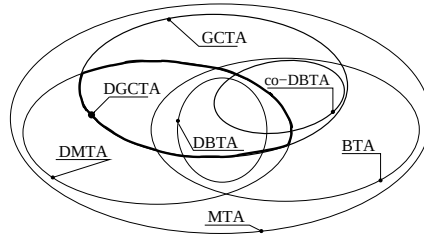


Fig. 4. Summary of the comparisons involving co-DBTA.

Lemma 9. $co\text{-}DBTA \subseteq GCTA \cap BTA$.

Proof. The inclusion $co\text{-}DBTA \subseteq GCTA \cap BTA$ is a direct consequence of Lemmas 7 and 8. To prove the strict inclusion, we can use the language $T = \{t \in T_\Sigma^\omega \mid t_0 \in T_1 \text{ and } t_1 \in T_1\}$, where T_1 is given in Figure 2. $\square$

Lemma 10.

1. (a) $co\text{-}DBTA \cap DBTA \neq \emptyset$;
 (b) $co\text{-}DBTA \cap (DMTA \setminus DBTA) \neq \emptyset$;
 (c) $co\text{-}DBTA \setminus DMTA \neq \emptyset$.
2. (a) $((BTA \cap GCTA \cap DMTA) \setminus DBTA) \not\subseteq co\text{-}DBTA$;
 (b) $((BTA \cap GCTA) \setminus DMTA) \not\subseteq co\text{-}DBTA$.

Proof. Consider first part 1. To prove statement (a), we use the language $T = \{t \in T_\Sigma^\omega \mid \forall x \in \pi_0, t(x) = a\}$. For statements (b) and (c), we respectively use T_3 and T_1 given in Figure 2.

Consider now part 2. To prove statement (a) we use $T = \{t \in T_\Sigma^\omega \mid t_0 \in T_3 \text{ and } t_1 \in T_3\}$. Finally, for statement (b) we use $T = \{t \in T \mid t_0 \in T_1 \text{ and } t_1 \in T_1\}$, where T_1 and T_3 are given in Figure 2. $\square$

6 Conclusion

In this paper, we have dealt with co-Büchi tree automata along with both alternating and generalized paradigms (AGCTA). These automata naturally characterize the complement of languages accepted by generalized Büchi tree automata.

Alternating tree automata were first considered by Muller and Schupp in [MS87]. They showed that while alternation does not give more expressive power to Muller, Rabin, Street and Büchi paradigms (*simulation theorem*), it gives more succinct automata. Once the simulation theorem holds, emptiness for alternating automata can be reduced to the emptiness problem for classical automata. For several classes of automata this procedure is optimal in the sense that it matches the known lower bounds. Here, we have proved that the simulation theorem does not hold for AGCTA. Thus, for this class it is not possible to check for the emptiness via a translation to generalized co-Büchi tree automata. However, using a reduction to parity automata, we have proved that the emptiness for AGCTA is in EXPTIME. This results has also been shown to be complete via a reduction from the emptiness problem for weak alternating Büchi tree automata.

We have also shown that the generalized framework does not give more acceptance power to this paradigm. In particular, given an AGCTA with n states and index k , it is possible to build a language equivalent ACTA with $\mathcal{O}(n(k+1))$ states. Although this language equivalence holds, the generalized one is still of interest since it can lead to more succinct automata.

For the class of languages whose complement is accepted by deterministic generalized Büchi tree automata, we have shown that the emptiness problem is decidable in quadratic time, using a linear characterization of this class of languages via generalized co-Büchi tree automata. In particular, given a deterministic generalized Büchi tree automaton A with n states and index k , we can check the emptiness for the complement of $T(A)$ in time $\mathcal{O}(nk)$.

Finally, we have fully compared the classes of languages whose complement is respectively accepted by deterministic and nondeterministic Büchi tree automata with the main classes studied in the literature, showing interesting relationships.

References

- [BN95] D. Beauquier and D. Niwinski. Automata on infinite trees with counting constraints. *Information and Computation*, 120:117–125, 1995.
- [Büc62] J.R. Büchi. On a decision method in restricted second-order arithmetic. In *Proceedings of the International Congress on Logic, Methodology, and Philosophy of Science 1960*, pages 1–12. Stanford University Press, 1962.
- [Cho74] Y. Choueka. Theories of automata on ω -tapes: a simplified approach. *Journal of Computer and System Sciences*, 8:117–141, 1974.
- [CKS81] A.K. Chandra, D.C. Kozen, and L.J. Stockmeyer. Alternation. *Journal of the ACM*, 28(1):114 – 133, 1981.
- [EJ88] E.A. Emerson and C.S. Jutla. The complexity of tree automata and logics of programs. In *Proceedings 29th Annual IEEE Symp. on Foundations of Computer Science, FOCS'88*, pages 328 – 337, 1988.

- [EJ91] E. A. Emerson and C. S. Jutla. Tree automata, μ -calculus and determinacy. In *Proceedings 32nd Annual IEEE Symp. on Foundations of Computer Science, FOCS'91*, pages 368–377, 1991.
- [EL85] E.A. Emerson and C.L. Lei. Modalities for model-checking: Branching time logic strikes back. In *Proceedings of the 12th ACM Symposium on Principles of Programming Languages*, pages 84–96, 1985.
- [GPVW95] R. Gerth, D. Peled, M.Y. Vardi, and P. Wolper. Simple on-the-fly automatic verification of linear temporal logic. In *Protocol Specification Testing and Verification*, pages 3 – 18. Chapman & Hall, 1995.
- [GTW02] E. Grädel, W. Thomas, and T. Wilke, editors. *Automata, Logics, and Infinite Games: A Guide to Current Research*, volume 2500 of *LNCS*, 2002.
- [Jur98] Marcin Jurdziński. Deciding the winner in parity games is in $UP \cap co-UP$. *Information Processing Letters*, 68(3):119–124, 1998.
- [Kur94] R.P. Kurshan. *Computer-aided Verification of Coordinating Processes: the automata-theoretic approach*. Princeton University Press, 1994.
- [KV04] O. Kupferman and M.Y. Vardi. From complementation to certification. In *10th International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, *LNCS*. Springer-Verlag, 2004.
- [KVW00] O. Kupferman, M.Y. Vardi, and P. Wolper. An automata-theoretic approach to branching-time model checking. *Journal of the ACM*, 47(2):312–360, 2000.
- [Lan69] L. H. Landweber. Decision problems for ω -automata. *Mathematical System Theory*, 3:376–384, 1969.
- [LMN02] S. La Torre, A. Murano, and M. Napoli. Weak muller acceptance condition for tree automata. In *3rd Workshop in Verification, Model Checking, and Abstract Interpretation, VMCAI 2002. LNCS*, volume 2294: 240–254, 2002.
- [McN66] R. McNaughton. Testing and generating infinite sequences by a finite automaton. *Information and Control*, 9:521–530, 1966.
- [MS87] D. E. Muller and P. E. Schupp. Alternating automata on infinite trees. *Theoretical Computer Science*, 54:267–276, 1987.
- [MS95] D. E. Muller and P. E. Schupp. Simulating alternating tree automata by nondeterministic automata: new results and proofs of theorems of Rabin, McNaughton, and Safra. *Theoretical Computer Science*, 141:69–107, 1995.
- [Rab69] M.O. Rabin. Decidability of second-order theories and automata on infinite trees. *Trans. Amer. Math. Soc.*, 141:1 – 35, 1969.
- [Rab70] M.O. Rabin. Weakly definable relations and special automata. *Mathematical Logic and Foundations of Set theory*, 1970.
- [Rab72] M.O. Rabin. Automata on infinite objects and church’s problem. *Trans. Amer. Math. Soc.*, 1972.
- [Sei90] H. Seidl. Deciding equivalence of finite tree automata. *SIAM Journal of Computing*, 19:424–437, 1990.
- [Tho90] W. Thomas. Automata on infinite objects. In J.van Leeuwen, editor, *Handbook of Theoretical Computer Science, Vol.B*, pages 133 – 191. 1990.
- [VW86] M.Y. Vardi and P. Wolper. Automata-theoretic techniques for modal logics of programs. *Journal of Computer and System Sciences*, 32:182 – 211, 1986.
- [VW94] M.Y. Vardi and P. Wolper. Reasoning about infinite computations. *Information and Computation*, 15:1 – 37, 1994.
- [Wil01] Thomas Wilke. Alternating tree automata, parity games, and modal μ -calculus. *Bull. Soc. Math. Belg.*, 8(2), May 2001.